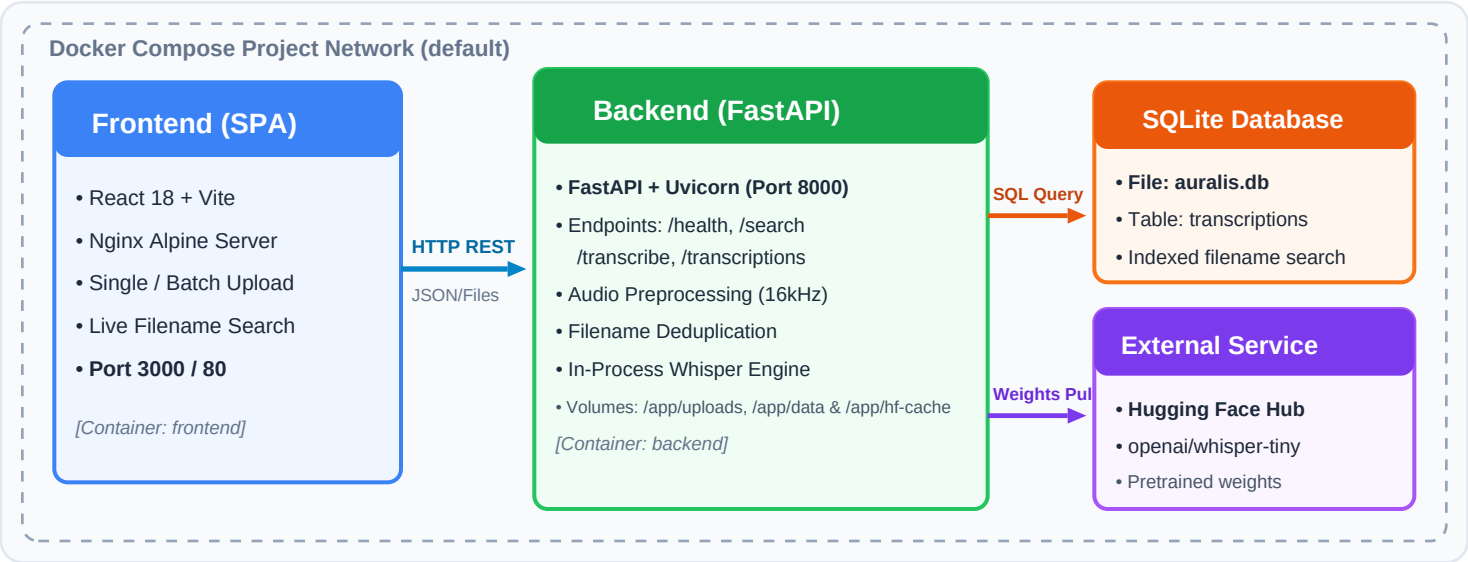


Auralis — System Architecture & Technical Specifications

Role: Software Engineer Technical Assessment | Project: Auralis Speech-to-Text Platform | Model: openai/whisper-tiny

1. System Architecture Diagram



2. Architectural Components Breakdown

a) Frontend (Single-Page Application):

- Stack & Build:** React 18, Vite build system, HTML5 Drag & Drop File API, CSS responsive styling.
- Capabilities:** Single & batch audio upload via drag-and-drop or file picker with a real-time upload progress bar, persistent transcription records table, and client-to-server filename search.
- Containerization:** Packaged with multi-stage Alpine Nginx serving static assets on port 80 (mapped to host 3000).

b) Backend (RESTful Web Service):

- Engine & Framework:** FastAPI (Python 3.11) with Uvicorn ASGI, asynchronous request handling, CPU-bound Whisper inference offloaded to a worker thread (asyncio.to_thread) so it doesn't block the event loop, and automatic OpenAPI/Swagger documentation.
- Endpoints:** GET /health (liveness probe), POST /transcribe (multipart single/batch ingestion with deduplication), GET /transcriptions (chronological record retrieval), GET /search?filename=... (indexed substring query).
- Audio Preprocessing Pipeline:** Decodes uploaded formats (MP3/WAV/M4A), resamples audio to 16,000 Hz, downmixes stereo to mono, and normalizes amplitudes to float32 tensors per Whisper specification.
- Filename Deduplication:** Prevents overwriting existing backend records by checking disk and SQLite; appends unique deterministic UUID suffix ({stem}_{uuid[:8]}{ext}) while preserving the original user-facing filename.

c) Database (SQLite):

- Storage:** SQLite 3 relational database (auralis.db) mounted on persistent Docker volume (backend-data).
- Schema & Indexing:** Table transcriptions stores id (PK), filename (UNIQUE), original_filename, transcript, and created_at (ISO 8601 UTC). B-Tree indexes on filename and original_filename.

d) External Services & Model Integration:

- Model Hub:** Hugging Face Hub (openai/whisper-tiny, ~151 MB weights). The model is pre-warmed during FastAPI's startup lifespan, before the container reports healthy, so the first real transcription request is never the one paying the load cost.
- Runtime Dependency:** The Hugging Face cache directory is mounted on a persistent Docker volume (backend-hf-cache), so weights are downloaded from Hugging Face Hub once and reused across container restarts and rebuilds; uploaded audio is always processed locally inside the backend container.

3. Assumptions and Design Considerations

A. Operational & Architectural Assumptions:

- 1. In-Process vs. Microservice Inference:** Whisper-tiny is executed directly within the FastAPI backend worker process. Given the compact model footprint (39M parameters, ~151MB RAM), in-process inference eliminates network overhead, simplifies deployment into a single backend container, and adheres strictly to the single-repo specification.
- 2. Single vs. Batch File Contract:** The POST /transcribe endpoint standardizes on returning an array of JSON objects with schema fields: id, filename, original_filename, transcript, and created_at.
- 3. Persistence & Storage Separation:** Audio files uploaded by users are saved in uploads/ while metadata and transcriptions reside in SQLite. Both directories are mounted via Docker named volumes to survive container restarts and rebuilds.
- 4. Threaded, Not Parallel, Inference:** Whisper inference is CPU-bound and blocking, so it's offloaded to a worker thread (asyncio.to_thread) per file rather than run inline in the async request handler. This keeps the single event loop free to serve unrelated concurrent requests (health checks, listings, other uploads) while a transcription is in progress. Files within one batch upload are still transcribed one at a time against the shared in-memory model; parallelizing them was deliberately left out of scope, since real speedup would need a bounded worker pool and is capped by the container's available CPU cores, not just code changes.

B. Reliability, Performance & Scalability Considerations:

Domain	Current Implementation	Production Scale Recommendation
Inference Concurrency	CPU inference per file, offloaded to a worker thread so it doesn't block the event loop for other requests; files within one batch upload still transcribe one at a time.	Offload transcription to an asynchronous Celery / Redis task queue with WebSocket status streaming to prevent HTTP timeouts on long audio files.
Storage Architecture	Local container filesystem mounted to Docker named volume.	Migrate audio storage to cloud object storage (AWS S3, GCP Cloud Storage) with presigned URLs for direct client uploads.
Database Scaling	Embedded SQLite with B-Tree indexes for fast local lookups.	Migrate to PostgreSQL or distributed relational database with full-text search (pgvector or Elasticsearch) for transcript content searching.
Security & Compliance	Path-safe filename handling, UUID collision isolation, and configurable CORS origins.	Add rate limiting (SlowAPI), JWT bearer authentication, audio malware scanning (ClamAV), and HTTPS TLS termination at reverse proxy.

4. Verification & Testing Matrix

- Backend Unit Tests:** 5 comprehensive pytest suites in backend/tests/test_api.py verifying /health, the API root info response, /transcriptions, substring search filtering, and unique filename collision generation.
- Frontend Unit Tests:** 4 Vitest/RTL tests in frontend/src/App.test.jsx verifying tabular transcription rendering, upload with progress reporting, drag-and-drop file selection, and live search query triggering.
- Sample Audio Evaluation:** Evaluated on the three provided sample MP3 files (Sample 1.mp3, Sample 2.mp3, Sample 3.mp3) confirming successful end-to-end ingestion and speech transcription.
- Local Setup:** Run setup-local.sh from the repository root to install backend and frontend dependencies, or run backend/setup-backend.sh to prepare only the Python backend environment.